

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Российский экономический университет им. Г.В. Плеханова»

Сопроводительная записка

по дисциплине «Поддержка и тестирование программных модулей»

на тему: Практическая работа №1

Специальность 09.02.04 «Информационные системы и программирование»

Исполнитель

_____/Д.Ю.Чащин /

Группа ИПо-22

«3» Октября 2026 г.

Пермь, 2026

Оглавление

1. Введение	2
2. Создание основного проекта BANK.....	3
3. Создание проекта модульных тестов BANKTESTS	4
4. Написание и запуск первого теста.....	5
5. Тестирование исключительных ситуаций	6
6. Рефакторинг кода и тестов.....	6
ЗАКЛЮЧЕНИЕ.....	7

1.Введение

Цель работы: освоение процесса разработки консольного приложения на языке C# в среде Visual Studio и приобретение практических навыков написания модульных тестов с использованием фреймворка MSTest.

Задачи:

1. Создать проект консольного приложения Bank с классом BankAccount.
2. Создать проект модульных тестов BankTests.
3. Написать тесты для проверки корректности работы метода Debit.
4. Выявить и исправить логическую ошибку в коде с помощью тестов.
5. Провести рефакторинг кода и тестов для повышения их надежности.

2.Создание основного проекта BANK

2.1. Создание проекта

В среде Visual Studio был создан проект по шаблону «**Консольное приложение на C#**» (.NET Core). Проекту присвоено имя Bank. Выбрана рекомендованная целевая платформа (.NET 6).

2.2. Исходный код класса BankAccount

Файл Program.cs был переименован в BankAccount.cs. В него помещен код, описывающий банковский счет. На данном этапе в методе Debit (списание средств) **намеренно оставлена ошибка**: вместо вычитания суммы (-=) выполняется сложение (+=).

```

using System;
namespace BankAccountNS
{
    /// <summary>
    /// Bank account demo class.
    /// </summary>
    Ссылка: 4
    public class BankAccount
    {
        Ссылка: 2
        private readonly string m_customerName;
        Ссылка: 5
        private double m_balance;
        Ссылка: 0
        private BankAccount() { }
        Ссылка: 1
        public BankAccount(string customerName, double balance)
        {
            m_customerName = customerName;
            m_balance = balance;
        }
        Ссылка: 0
        public string CustomerName
        {
            get { return m_customerName; }
        }
        Ссылка: 1
        public double Balance
        {
            get { return m_balance; }
        }
        Ссылка: 1
        public void Debit(double amount)
        {
            if (amount > m_balance)
            {
                throw new ArgumentOutOfRangeException("amount");
            }
            if (amount < 0)
            {
                throw new ArgumentOutOfRangeException("amount");
            }
            m_balance += amount; // intentionally incorrect code
        }

        public void Credit(double amount)
        {
            if (amount < 0)
            {
                throw new ArgumentOutOfRangeException("amount");
            }
            m_balance += amount;
        }
        Ссылка: 0
        public static void Main()
        {
            BankAccount ba = new BankAccount("Mr. Bryan Walton",
            11.99);
            ba.Credit(5.77);
            ba.Debit(11.22);
            Console.WriteLine("Current balance is ${0}", ba.Balance);
        }
    }
}

```

3. Создание проекта модульных тестов BANKTESTS

3.1. Добавление проекта тестов

В обозревателе решений через контекстное меню решения было выбрано «Добавить» → «Создать проект». Выбран шаблон «Проект модульного теста MSTest (.NET Core)» на языке C#. Проект назван BankTests.

3.2. Добавление ссылки на проект Bank

В проекте BankTests через узел «Зависимости» добавлена ссылка на проект Bank (команда «Добавить ссылку на проект»).

3.3. Переименование файла и класса

Файл UnitTest1.cs переименован в BankAccountTests.cs. Класс внутри файла также переименован в BankAccountTests. В начало файла добавлена директива using BankAccountNS;.

Исходный код файла после переименования:

```
namespace BankTestsBankTests
{
    [TestClass]
    Ссылка: 0
    public sealed class BankAccountTests
    {
        [TestMethod]
        Ссылка: 0
        public void TestMethod1()
        {
        }
    }
}
```

4. Написание и запуск первого теста

4.1. Создание метода теста

Стандартный метод TestMethod1 был удален. В класс BankAccountTests добавлен метод Debit_WithValidAmount_UpdatesBalance, проверяющий корректность списания допустимой суммы.

```
[TestMethod]
Ссылка: 0
public void Debit_WithValidAmount_UpdatesBalance()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = 4.55;
    double expected = 7.44;
    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

    // Act
    account.Debit(debitAmount);

    // Assert
    double actual = account.Balance;
    Assert.AreEqual(expected, actual, 0.001, "Account not debited correctly");
}
```

4.2. Запуск теста и обнаружение ошибки

При запуске тестов (Ctrl + R, V) тест **провалился**. Сообщение об ошибке

показало, что баланс не уменьшился, а увеличился, что подтвердило наличие логической ошибки в методе Debit.

4.3. Исправление ошибки

В файле BankAccount.cs строка `m_balance += amount;` была заменена на `m_balance -= amount;`. Повторный запуск тестов показал успешное прохождение.

5. Тестирование исключительных ситуаций

5.1. Тест на отрицательную сумму

Добавлен тест, проверяющий, что при попытке списания отрицательной суммы выбрасывается исключение `ArgumentOutOfRangeException`.

```
[TestMethod]
Ссылка 0
public void Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRangeException()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = -100.00;
    BankAccount account = new BankAccount("Mr. Bryan Walton",
beginningBalance);
    // Act and assert
    Assert.ThrowsExactly<System.ArgumentOutOfRangeException>(() =>
account.Debit(debitAmount));
}
```

5.2. Тест на превышение баланса

Создан аналогичный тест

`Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRangeException`, в котором сумма списания (20.0) превышает баланс (11.99).

6. Рефакторинг кода и тестов

6.1. Рефакторинг класса BankAccount

В класс `BankAccount` добавлены строковые константы для сообщений об ошибках. Условия в методе `Debit` изменены для передачи этих сообщений в конструктор исключения.

```
public const string DebitAmountExceedsBalanceMessage = "Debit amount exceeds balance";
```

```
public const string DebitAmountLessThanZeroMessage = "Debit amount is less than zero";
```

Обновленный метод `Debit`:

```

if (amount > m_balance)
{
    throw new System.ArgumentOutOfRangeException("amount", amount, DebitAmountExceedsBalanceMessage);
}

if (amount < 0)
{
    throw new System.ArgumentOutOfRangeException("amount", amount, DebitAmountLessThanZeroMessage);
}

m_balance -= amount; // intentionally incorrect code

```

6.2. Рефакторинг тестового метода

Для проверки конкретного сообщения об ошибке была использована конструкция try/catch с методом StringAssert.Contains. Также добавлен вызов Assert.Fail() на случай, если исключение не будет выброшено

```

[TestMethod]
Ссылка: 0
public void Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRangeException()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = 20.0;
    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

    // Act
    try
    {
        account.Debit(debitAmount);
    }
    catch (System.ArgumentOutOfRangeException e)
    {
        // Assert
        StringAssert.Contains(e.Message, BankAccount.DebitAmountExceedsBalanceMessage);
        return;
    }

    Assert.Fail("The expected exception was not thrown.");
}

```

ЗАКЛЮЧЕНИЕ

В ходе работы были освоены основные приемы модульного тестирования в Visual Studio. С помощью тестов была обнаружена и исправлена логическая ошибка в методе списания средств. Проведен рефакторинг кода и тестов, что сделало сообщения об ошибках более информативными, а сами тесты — более надежными. Полученные навыки позволяют создавать более качественный и устойчивый к ошибкам код.